```
!pip install optuna
import optuna
from time import time
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.svm import SVR
from sklearn.kernel_ridge import KernelRidge
from sklearn.gaussian_process import GaussianProcessRegressor

from sklearn.gaussian_process.kernels import RBF, Matern, DotProduct, WhiteKernel

from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
from sklearn.metrics import mean_absolute_error

from sklearn.metrics import accuracy_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

```
Requirement already satisfied: optuna in /usr/local/lib/python3.12/dist-packages (4.7.0)
Requirement already satisfied: alembic>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from optuna) (1.18.4)
Requirement already satisfied: colorlog in /usr/local/lib/python3.12/dist-packages (from optuna) (6.10.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from optuna) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from optuna) (26.0)
Requirement already satisfied: sqlalchemy>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from optuna) (2.0.47)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from optuna) (4.67.3)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.12/dist-packages (from optuna) (6.0.3)
Requirement already satisfied: Mako in /usr/local/lib/python3.12/dist-packages (from alembic>=1.5.0->optuna) (1.3.10)
Requirement already satisfied: typing-extensions>=4.12 in /usr/local/lib/python3.12/dist-packages (from alembic>=1.5.0->optuna)
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-packages (from sqlalchemy>=1.4.2->optuna) (3.3.2)
Requirement already satisfied: MarkupSafe>=0.9.2 in /usr/local/lib/python3.12/dist-packages (from Mako->alembic>=1.5.0->optuna)
```

```
from google.colab import files
uploaded = files.upload()
df = pd.read_csv("winequality-red.csv")
df.head()
```

Choose Files winequality-red.csv

winequality-red.csv(text/csv) - 85746 bytes, last modified: 3/6/2026 - 100% done

```
Saving winequality-red.csv to winequality-red (1).csv
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6

Next steps: Generate code with df New interactive sheet

```
X = df.drop("quality", axis=1)
y = df["quality"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Check shapes
print("Train data shape:", X_train.shape)
print("Test data shape:", X_test.shape)
```

```
Train data shape: (1119, 11)
Test data shape: (480, 11)
```

```
def create_model(trial):
    model_type = trial.suggest_categorical('model_type',['Logistic-regression', 'SVM'])

    if model_type == 'SVM':
        kernel = trial.suggest_categorical('SVM-kernel',['linear','poly','rbf','sigmoid'])
        C = trial.suggest_float('SVM-C',0.1, 1000,log=True)
        gamma = trial.suggest_float('SVM-gamma',0.0001, 10,log=True)
        model = SVC(kernel=kernel,C=C,gamma=gamma)

    if model_type == 'Logistic-regression':
        penalty = trial.suggest_categorical('penalty',['l1','l2'])
        if penalty == 'l1':
            solver = 'saga'
        else:
            solver = 'lbfgs'
        regularization = trial.suggest_float('Logistic-regularization',0.01, 100,log=True)
        model = LogisticRegression(penalty=penalty,C=regularization,solver=solver,max_iter=5000)

    if trial.should_prune():
        raise optuna.TrialPruned()

    return model
```

```
def objective(trial):
    model = create_model(trial)
    scores = cross_val_score(model,X_train,y_train,cv=5,scoring="accuracy")
    return np.mean(scores)
```

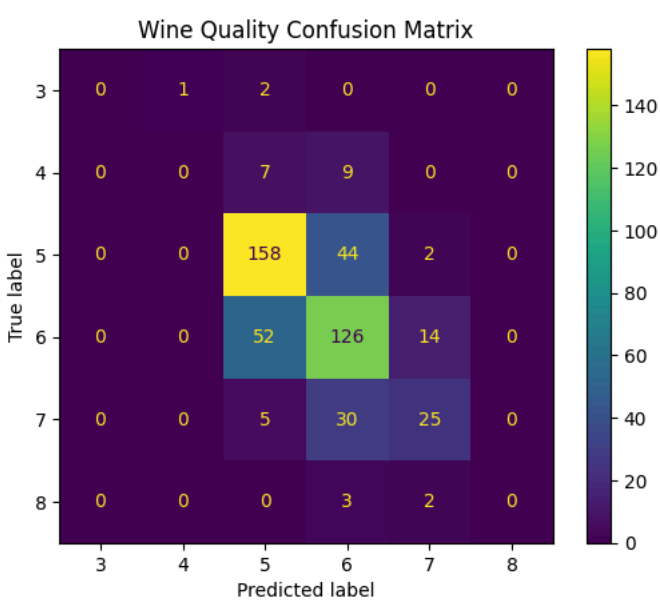
```
start = time()

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=100)

best_params = study.best_params

print("\nBest parameters:", best_params)
```

```
[I 2026-03-06 11:18:35,281] Trial 83 finished with value: 0.6032270980140936 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:18:35,629] Trial 84 finished with value: 0.6014093529788598 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:18:36,147] Trial 85 finished with value: 0.6085682254964766 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:18:47,608] Trial 86 finished with value: 0.5844650864830238 and parameters: {'model_type': 'Logistic-regressi
[I 2026-03-06 11:18:48,155] Trial 87 finished with value: 0.5782391095451633 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:18:48,476] Trial 88 finished with value: 0.6005285073670723 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:18:48,808] Trial 89 finished with value: 0.42631726457399105 and parameters: {'model_type': 'SVM', 'SVM-kerne
[I 2026-03-06 11:18:49,240] Trial 90 finished with value: 0.6166279628443305 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:18:49,701] Trial 91 finished with value: 0.6166319666880204 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:18:50,430] Trial 92 finished with value: 0.6166399743754004 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:18:51,078] Trial 93 finished with value: 0.6139333760409993 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:18:52,027] Trial 94 finished with value: 0.5969370595771941 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:18:52,771] Trial 95 finished with value: 0.6166359705317104 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:18:53,207] Trial 96 finished with value: 0.6139413837283791 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:19:03,687] Trial 97 finished with value: 0.5844650864830238 and parameters: {'model_type': 'Logistic-regressi
[I 2026-03-06 11:19:04,367] Trial 98 finished with value: 0.6148342408712365 and parameters: {'model_type': 'SVM', 'SVM-kernel
[I 2026-03-06 11:19:05,000] Trial 99 finished with value: 0.6157230941704036 and parameters: {'model_type': 'SVM', 'SVM-kernel

Best parameters: {'model_type': 'SVM', 'SVM-kernel': 'rbf', 'SVM-C': 1.2621277876554782, 'SVM-gamma': 0.4768862861372544}
```

```
best_model = create_model(study.best_trial)
best_model.fit(X_train, y_train)
scores = cross_val_score(best_model, X_train, y_train, cv=5)

print("Best CV accuracy:", np.mean(scores))

y_pred = best_model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average="weighted")

print("Test Accuracy:", acc)
print("Test F1-score:", f1)
```

```
Best CV accuracy: 0.6166399743754004
Test Accuracy: 0.64375
Test F1-score: 0.6239696088745069
```

```
labels = np.unique(y)

cfm = confusion_matrix(y_test, y_pred, labels=labels)
cm_display = ConfusionMatrixDisplay(confusion_matrix=cfm,display_labels=labels)
cm_display.plot()
plt.title("Wine Quality Confusion Matrix")
plt.show()
```




```
#for letter b
def create_model(trial):
    model_type = trial.suggest_categorical('model_type',['SVR', 'KRR', 'GPR'])
    if model_type == 'SVR':
        kernel = trial.suggest_categorical('SVR-kernel',['linear','poly','rbf','sigmoid'])
        C = trial.suggest_float('SVR-C',0.1, 100,log=True)
```

```
        gamma = trial.suggest_float('SVR-gamma',0.0001, 10,log=True)
        epsilon = trial.suggest_float('SVR-epsilon',0.001, 1,log=True)
        model = SVR(kernel=kernel,C=C,gamma=gamma,epsilon=epsilon)

    if model_type == 'KRR':
        alpha = trial.suggest_float('KRR-alpha',0.001, 10,log=True)
        kernel = trial.suggest_categorical('KRR-kernel',['linear','poly','rbf'])
        model = KernelRidge(alpha=alpha,kernel=kernel)

    if model_type == 'GPR':
        alpha = trial.suggest_float('GPR-alpha',1e-5, 1,log=True)
        kernel_type = trial.suggest_categorical('GPR-kernel',['rbf','matern','dot'])
        if kernel_type == 'rbf':
            kernel = RBF()
        if kernel_type == 'matern':
            kernel = Matern()
        if kernel_type == 'dot':
            kernel = DotProduct()
        kernel = kernel + WhiteKernel()
        model = GaussianProcessRegressor(kernel=kernel,alpha=alpha)

    if trial.should_prune():
        raise optuna.TrialPruned()

    return model
```

```
def objective(trial):
    model = create_model(trial)
    scores = cross_val_score(model,X_train,y_train,cv=5,scoring='neg_mean_squared_error')
    return np.mean(scores)
```

```
start = time()

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=100)
best_params = study.best_params

print("\nBest parameters:", best_params)
```

```
[I 2026-03-06 11:42:36,464] Trial 95 finished with value: -0.41392741393213883 and parameters: {'model_type': 'GPR', 'GPR-alpha
/usr/local/lib/python3.12/dist-packages/sklearn/gaussian_process/kernels.py:442: ConvergenceWarning: The optimal value found fo
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/gaussian_process/kernels.py:442: ConvergenceWarning: The optimal value found fo
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/gaussian_process/kernels.py:442: ConvergenceWarning: The optimal value found fo
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/gaussian_process/kernels.py:442: ConvergenceWarning: The optimal value found fo
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/gaussian_process/kernels.py:442: ConvergenceWarning: The optimal value found fo
  warnings.warn(
[I 2026-03-06 11:43:01,741] Trial 96 finished with value: -0.4223397106276926 and parameters: {'model_type': 'GPR', 'GPR-alpha
[I 2026-03-06 11:43:16,176] Trial 97 finished with value: -0.4310984183615754 and parameters: {'model_type': 'GPR', 'GPR-alpha
[I 2026-03-06 11:43:33,338] Trial 98 finished with value: -0.41392741609666495 and parameters: {'model_type': 'GPR', 'GPR-alpha
[I 2026-03-06 11:43:33,758] Trial 99 finished with value: -32.68099881392005 and parameters: {'model_type': 'KRR', 'KRR-alpha'

Best parameters: {'model type': 'GPR', 'GPR-alpha': 0.3546547773897675, 'GPR-kernel': 'matern'}
```

```
best_model = create_model(study.best_trial)
best_model.fit(X_train, y_train)
scores = cross_val_score(best_model,X_train,y_train,cv=5,scoring='neg_mean_squared_error')
print("Best CV MSE:", -np.mean(scores))
```

```
Best CV MSE: 0.41392733185301755
```

```
y_pred = best_model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
mad = mean_absolute_error(y_test, y_pred)

print("Test MSE:", mse)
print("Test R2:", r2)
print("Test MAD:", mad)
```

```
Test MSE: 0.37463326979627537
Test R2: 0.4221556126456112
Test MAD: 0.4683005222256078
```

No, the model did not improve upon the results reported in the paper. The best model in the paper achieved a MAD of approximately 0.46 for the red wine dataset, while the best model obtained in this experiment produced a MAD of 0.4683. Although the performance is very close, the error is slightly higher.

If treated as classification which predicted discrete values for quality of wine, the model achieved a test accuracy of 61.66% F1 score of (0.623). If treated as regression, it will predict continouos value for wine quality which can allow more precise estimates. My values were MSE = 0.3746 and MAD = 0.4683, suggesting that the predicted wine quality by the model is less than a half point away from actual wine rating.

In general classification is easier if we need to label things to categories, while regression allows to look into small differences in values, which can be better for detailed analysis.